SENTIMENT ANALYSIS AND STANCE DETECTION OF PVV-RELATED REDDIT COMMENTS IN THE 2023 DUTCH GENERAL ELECTION USING MACHINE LEARNING

QIAN HUANG

THESIS SUBMITTED IN PARTIAL FULFILLMENT
OF THE REQUIREMENTS FOR THE DEGREE OF
MASTER OF SCIENCE IN DATA SCIENCE & SOCIETY
AT THE SCHOOL OF HUMANITIES AND DIGITAL SCIENCES
OF TILBURG UNIVERSITY

STUDENT NUMBER

2109043

COMMITTEE

Dr. Fred Blain
Dr. Marijn van Wingerden

LOCATION

Tilburg University
School of Humanities and Digital Sciences
Department of Cognitive Science &
Artificial Intelligence
Tilburg, The Netherlands

DATE

July 8, 2024

WORD COUNT

8792

ACKNOWLEDGMENTS

The process of writing and revising the thesis was very stressful. I am deeply grateful to Marijn van Wingerden for his exceptional patience, diligent and responsible attitude during our thesis group meetings, his detailed suggestions, his feedback, and his timely responses to my questions. I would also like to thank my supervisor, Fred Blain, for his comments and inspection of my thesis. Additionally, I am very grateful to my family, especially my uncle, for their unwavering support. Even from afar in China, my family could sense when something was wrong during our video calls and provided timely psychological counseling and encouragement throughout the master.

SENTIMENT ANALYSIS AND STANCE DETECTION OF PVV-RELATED REDDIT COMMENTS IN THE 2023 DUTCH GENERAL ELECTION USING MACHINE LEARNING

QIAN HUANG

Abstract

In the 2023 Dutch general election, the far-right political party, the Party for Freedom (PVV), won, marking a historic moment in Dutch politics. Currently, there is no published literature exploring the Dutch general election using machine learning in the field of Natural Language Processing (NLP). To fill this gap, this study conducts sentiment analysis and stance detection on Reddit comments related to the PVV in the context of the 2023 election using machine learning. A dataset in Dutch containing comments related to PVV was obtained from Reddit and manually labeled with sentiment and stance labels according to consistent guidelines by two Dutch speakers to establish the ground truth. This work employs Logistic Regression (LR), Random Forest (RF), and eXtreme Gradient Boosting (XGBoost) models, along with TF-IDF, word2vec, and FastText feature extraction methods, to compare their performance with the ground truth. Additionally, the impact of the data augmentation technique Easy Data Augmentation (EDA) on model performance was evaluated. We also explore whether adding sentiment labels in stance detection and stance labels in sentiment analysis as additional features can further improve model performance. The study finds that although the overall model performance can achieve comparable outcomes to the ground truth, it is not ideal, with average macro F1 scores far lower than 100%. Word2vec embeddings generally outperform TF-IDF and FastText. However, the EDA technique is unstable, sometimes reducing rather than improving model performance, and models with EDA exhibit worse generalization ability compared to those without EDA. The best sentiment analysis model on the test set is RF with word2vec without EDA (F1 score: 46.48%), and the best stance detection model is XGBoost with word2vec without EDA (F1 score: 46.49%). Both models showed biases: the sentiment model against the Neither stance and the stance model against the Positive sentiment. Adding stance labels as features in sentiment analysis and sentiment labels as features in stance detection can improve model performance, but these improvements were not statistically significant. Due to the low inter-rater reliability achieved in data annotation, which was 36.10% for sentiment and 41.25% for stance in the training set, this study cannot draw any tentative conclusions. Overall, this study contributes to the exploration of the Dutch election in the NLP field using machine learning, providing a deeper understanding of public sentiment and stance behind the rise of far-right forces through the analysis of political texts.

1 DATA SOURCE, ETHICS, CODE, AND TECHNOLOGY STATEMENT

The dataset was obtained from Reddit using the official API tool. The data does not contain any personal information, and this project does not involve collecting data from human participants. All figures, tables, and images used in the thesis are produced by the author. ChatGPT was used to debug the code and check the grammar and syntax errors of the thesis to ensure clarity. The data and code for the thesis can be found here: https://github.com/QianNetherladns/Thesis-. The API credentials are as follows:

```
client_id = "ugRlNzKHhKM-lu5Kd4I1iQ"
client_secret = "0Cju4Qw1wf77GGy9WAZ9-_RePexrIg"
user_agent = "YourUniqueUserAgent v1.0"
```

Appendix D displays a sample of labeled train data before data cleaning to confirm that no sensitive information was collected.

2 INTRODUCTION

2.1 *Problem Statement*

After the Dutch general elections of 2023, the political landscape of the Netherlands underwent a major shift. At the time of writing, PVV, known for its controversial positions, had successfully formed a coalition government and appointed a new prime minister. This historic shift marked a turning point in Dutch politics. The PVV is a prominent right-wing party in the Dutch government, advocating for the Netherlands to exit the European Union (Nexit) (Otjes, 2022). The PVV's key figure, Geert Wilders, is known for his fierce criticism of Islam (Van Heerden & Creusen, 2014). The victory represents the rise of right-wing forces, which could have far-reaching implications for economic policy, immigration policy, trade policy, cultural protectionism, and environmental policy (Rodrik, 2021). This development deserves close attention and study by all sectors (Rodrik, 2021).

Sentiment analysis technology has a long history in the field of natural language processing (NLP) and has become a standard tool for analyzing political texts (Bestvater & Monroe, 2023). However, most political sentiment analysis research overlooks public stance (Chaudhry et al., 2021; Hidayatullah et al., 2021; Schmidt et al., 2022). The difference between sentiment analysis and stance detection is that sentiment analysis determines the overall tone of a text, while stance detection identifies the text's attitude towards a specific target (S. Mohammad et al., 2016). In political texts, the attitude towards specific targets is usually of interest (Bestvater & Monroe, 2023). Moreover, political texts are complex and multifaceted. For movie or product reviews, positive and negative sentiments can directly reflect people's preferences and stances (Bestvater & Monroe, 2023). For political texts, relying solely on the sentiment of the text sometimes cannot accurately reflect its stance (Bestvater & Monroe, 2023). Therefore, analyzing political documents solely from the sentiment perspective is insufficient and could potentially lead to misleading results (Bestvater & Monroe, 2023). To provide a more comprehensive and accurate analysis of political phenomena, we choose to study the PVV party in the 2023 Dutch elections from the perspectives of sentiment and stance, which also contributes to the study of downstream political phenomena (Bestvater & Monroe, 2023).

Regarding the Dutch election, most literature is explanatory and descriptive (Belanger & Aarts, 2006; Emanuele et al., 2018; Otjes, 2022; Van Holsteyn, 2018; Vliegenthart, 2012). To the best of my knowledge, there is currently no research exploring the Dutch election using machine learning in the NLP field. Therefore, this paper aims to fill that gap.

We conduct sentiment analysis and stance detection separately using machine learning. The primary challenges of these tasks are the small and imbalanced labeled dataset, as well as the limited resources available in Dutch compared to English. To address these issues, we apply Easy Data Augmentation (EDA) to expand the dataset and address the imbalance problem, evaluating its performance. This is followed by out-of-sample evaluation and error analysis on the best-performing models. Additionally, beyond using only the text itself as input features, we added stance labels as features in sentiment analysis models and sentiment labels as features in stance detection models. This additional experiment was performed on the best-performing models without EDA to examine the effect of these additional features on model performance and to explore the relationship between sentiment and stance.

2.2 *Research Question*

The proposed research question is as follows:

> *To what extent can machine learning predict the sentiment and stance on PVV-related Reddit comments in the 2023 Dutch general election?*

The sub-questions are as follows:

RQ1 *How do Logistic Regression, Random Forest, and XGBoost with TF-IDF, word2vec, and FastText perform in sentiment analysis and stance detection compared to the ground truth?*

RQ2 *What is the impact of Easy Data Augmentation (EDA) on the performance of sentiment analysis and stance detection models?*

RQ3 *Do the top-performing models with EDA demonstrate better generalization ability compared to those without EDA?*

RQ4 *Does the sentiment analysis model with the best generalization ability perform similarly across support, against, and neither stances?*

RQ5 *Does the stance detection model with the best generalization ability perform similarly across positive, negative, and neither sentiments?*

RQ6 *Will adding stance labels as additional features in sentiment analysis and adding sentiment labels as additional features in stance detection improve model performance?*

2.3 *Societal and Scientific Relevance*

In recent years, the rise of right-wing forces has raised concerns about whether the existing political order can withstand the global surge of right-wing populism (Van Holsteyn, 2018). In this context, understanding the citizens' sentiment and stance towards PVV becomes crucial. This research provides a timely investigation into the sentiments and stances regarding the Dutch political shift, offering valuable insights into the public's perception of PVV's rise to power.

This study is the first to apply machine learning to the Dutch election in the field of NLP. It also advances the exploration of sentiment analysis and stance detection in NLP and political text analysis. Additionally, this study establishes a robust framework for future political sentiment analysis and stance detection, while contributing to research on the Dutch language.

3 RELATED WORK

3.1 *Dutch Election*

Belanger and Aarts (2006) explains the rise of the Dutch party List Pim Fortuyn in the 2002 Dutch election. Vliegenthart (2012) explores the professionalization of political communication by analyzing Dutch election campaign posters. Emanuele et al. (2018) examines the current issues before the next Dutch general election. Van Holsteyn (2018) studies the background, process, and outcomes of the 2017 Dutch election. Otjes (2022) investigates the 2021 Dutch election and its impact on the Netherlands' position in EU politics. These studies are mostly descriptive and explanatory. In the field of NLP, only one study on the Dutch elections has been found: Korbee (2021), which uses lexicon-based approaches to perform sentiment analysis on the 2021 Dutch general election. TextBlob (Loria et al., 2018) achieved the highest F1 score, with a value of 0.41, while the combination of TextBlob, SentiStrength (Thelwall et al., 2010), and Polyglot (Al-Rfou et al., 2013) achieved the highest precision, with a value of 0.45.

3.2 *Sentiment Analysis*

Although there is little published research on sentiment analysis for Dutch elections, sentiment analysis itself has been extensively studied, with machine learning being a primary approach (Cambria et al., 2017). Chaudhry et al. (2021) explored the sentiment of the 2020 US election using Naive Bayes (NB), achieving 94.58% accuracy. Rustam et al. (2021) utilized Extra Trees Classifiers (ETC), achieving an accuracy of 0.93 for sentiment analysis of a COVID-19 Twitter dataset. Elghazaly et al. (2016) used Support Vector Machine (SVM) and NB to analyze the Egypt election, with SVM achieving an F1 score of 0.871 and NB achieving an F1 score of 0.922. Hidayatullah et al. (2021) conducted sentiment analysis on the 2019 Indonesian presidential election using both machine learning and deep learning. Although Bi-directional Long Short-Term Memory (BiLSTM) performed best with 84.6% accuracy, machine learning algorithms showed competitive results, with SVM achieving 84.04% and LR achieving 83.15% accuracy. These results indicate that machine learning can achieve promising and competitive results in political text sentiment analysis.

Deep learning is another primary approach in sentiment analysis (Cambria et al., 2017; Yue & Jing, 2022) and achieves state-of-the-art results in many fields (Feng et al., 2019). RobBERT (Delobelle et al., 2020), a transformer-based model specifically trained for the Dutch language, has been proven to achieve state-of-the-art results in sentiment analysis with

95.144% accuracy on the Dutch Book Reviews dataset. However, deep learning algorithms require a large amount of data for training to achieve excellent results (Gong et al., 2022). In our research, we choose machine learning algorithms.

3.3 *Stance Detection*

As our study also delves into stance detection, this section covers work related to stance detection. Stance detection, as a NLP problem, is still in its infancy (Küçük & Can, 2020). S. Mohammad et al. (2016) organized the first well-known competition on Twitter stance detection in 2016. Subtask A was designed for supervised stance detection, involving the detection of stance towards five targets, covering topics from politics to societal and religious issues, with provided labeled training data. Zarrella and Marsh (2016) proposed a deep learning model, ranking first in Subtask A with an overall F1 score of 67.82% among 19 participants. W. Wei et al. (2016) used a Convolutional Neural Network (CNN), ranking second in Subtask A with an overall F1 score of 67.33%. However, their results did not surpass the baseline, which was a SVM with n-grams, achieving an overall F1 score of 68.98%, indicating the promise of machine learning in this field (Alturayeif et al., 2023). According to Küçük and Can (2020), SVM, LR, and decision tree classifiers are the most frequent supervised machine learning classifiers used in the competition. In our research, we choose LR, RF, and XGBoost algorithms.

3.4 *Sentiment and Stance in a single work*

This section covers research that combines sentiment and stance detection, with few studies addressing both. S. M. Mohammad et al. (2017) compared the impact of various features on sentiment analysis and stance detection using the SemEval Subtask A dataset (S. Mohammad et al., 2016), including n-grams and sentiment lexicons, in models employing SVM. Their results showed that SVM with n-grams outperformed the majority baseline in both tasks, and adding sentiment lexicons improved sentiment analysis performance from 73.3% to 78.9%. However, other additional features did not enhance performance.

Similarly, Buytaert (2018) used n-grams and sentiment lexicons to explore the effectiveness of machine learning in sentiment and stance analysis on political texts about gun control in the US. Küçük and Arıcı (2024) investigated machine learning algorithms for sentiment and stance analysis regarding COVID-19 vaccination but did not compare performance before and after adding stance and sentiment features.

Bestvater and Monroe (2023) distinguished between sentiment and stance in political texts regarding the Women's March, Trump, and judicial scandals, using SVM and BERT models for their analysis. They found that applying a trained sentiment model to stance detection (and vice versa) reduced performance, highlighting the conceptual difference between sentiment and stance. For instance, the sentiment-trained BERT achieved an F1 score of 0.954 on judicial scandal texts, but this score dropped to 0.582 when performing stance detection.

Unlike these studies that use sentiment lexicon features, we employ sentiment and stance labels as additional features to determine if they could further improve model performance.

3.5 *Feature Extraction Methods*

Feature extraction plays a crucial role in improving classification efficiency (Kokab et al., 2022; Yue & Jing, 2022). TF-IDF is frequently used with machine learning algorithms (Chaudhry et al., 2021; Dang et al., 2020; Elghazaly et al., 2016; Hidayatullah et al., 2021; Rustam et al., 2021; Zotova et al., 2020). However, TF-IDF fails to capture the semantic relationships between words (Prakoso et al., 2021). The advent of word embeddings revolutionized NLP tasks by capturing syntactic and semantic meaning simultaneously, providing crucial input features for classifiers (Chersoni et al., 2021). Dang et al. (2020) found that word2vec outperforms TF-IDF across various evaluation metrics, including accuracy, recall, precision, F1 score, and AUC, when applied to deep learning sentiment analysis on eight datasets covering movie reviews, product reviews, user opinions, book reviews, and music reviews. Most current research utilizes word2vec or Glove to obtain word vectors (Yue & Jing, 2022).

Wang et al. (2020) conducted a comprehensive comparison between contextual embeddings (BERT, ELMo) and classical embeddings (word2vec, Glove, FastText) using deep learning on text classification tasks. They found within classical embeddings, Glove outperforms the other two (Wang et al., 2020). Although Glove has shown outstanding performance, to the best of our knowledge, there are no pre-trained Glove embeddings available in Dutch.

Instead, FastText embeddings demonstrate competitive advantages by effectively handling n-grams and unknown words (da Costa et al., 2023). Badri et al. (2022) used Glove and FastText word embeddings as input features for hate speech detection, achieving a 90% F1 Score. Mojumder et al. (2020) utilized FastText embeddings for Bangla news classification, achieving 87.87% accuracy with BiLSTM. In our work, we use TF-IDF, word2vec, and FastText as feature extraction methods.

3.6 Data Augmentation in NLP

High performance in many NLP tasks often depends on the size and quality of the data, and overfitting tends to be more severe when training on smaller datasets (J. Wei & Zou, 2019). However, obtaining a large amount of labeled data is time-consuming and expensive (J. Chen et al., 2023). Data augmentation is an approach to solving the limited data problem while simultaneously reducing overfitting (Khosla & Saini, 2020).

J. Wei and Zou (2019) propose the Easy Data Augmentation (EDA) technique, which combines four token-level operations: randomly deleting a word in the sentence (Iyyer et al., 2015; Miao et al., 2020), randomly swapping two words (Artetxe et al., 2018; J. Wei & Zou, 2019), replacing a random word with its synonym (X. Zhang et al., 2015), and randomly inserting the synonym of a word (Pradana et al., 2023; J. Wei & Zou, 2019). J. Chen et al. (2023) define token-level augmentation as the local modification of words or phrases in sentences to expand data size. They find that word replacement consistently improve performance the most for supervised learning in news and topic classification tasks. However, Kumar et al. (2020) find that the semantic meaning of data augmented by EDA does not necessarily align with the original data, based on their analysis of augmented datasets from movie reviews, various intents, and different question types.

According to J. Wei and Zou (2019), EDA is more effective on smaller training sets, achieving an average performance increase of 3.0% on a training set with 500 samples and an average performance increase of 0.8% on the full dataset. Wirawan, Cahyono, et al. (2023) used EDA to expand data in sentiment analysis of cyberbullying, improving the state-of-the-art performance by 2.52%.

Since our labeled dataset has a sample size smaller than 500 and EDA is easier to implement (J. Wei & Zou, 2019), we choose EDA as the data augmentation technique and evaluate its performance.

4 METHODOLOGY

4.1 Dataset Description

We chose Reddit as the social media platform to obtain the data because it is renowned for its high traffic on the internet (Melton et al., 2021). This platform is prominently used by users for the anonymous contribution and discussion of political content (Soliman et al., 2019). Reddit features communities, known as subreddits, created by users. Each community follows specific rules set by its members (Melton et al., 2021).

Using the keyword "PVV," we selected posts and comments from various subreddits that are known for active discussions on Dutch politics and have significant participation from Dutch users in the general election period. The dataset is fully anonymized, does not include private information, and consists of 14,530 comments. We filtered out English comments and removed duplicates, focusing on Dutch-language content. Due to the nature of the Reddit platform, there are nested comments, so we split them. We then used the keywords 'pvv', 'PVV', 'Geert Wilders', 'Geert', 'Wilders', 'geert wilders', 'geert', and 'wilders' to select relevant comments to reduce noise for analysis. Finally, we removed comments that are less than five words long. After these steps, 3,940 comments remained.

4.2 *Dataset Preparation*

4.2.1 *Hand-labelling Procedure*

As the dataset is scraped from Reddit, it is inherently unlabeled. We selected 10% of the comments for training and another 10% for testing. Two native Dutch speakers were chosen to annotate the data with sentiment and stance labels under the same guideline using S. Mohammad et al. (2016) as a reference, which can be seen in Appendix B. A third person, the author, confirmed the labels for both the training and testing datasets as the final decision.

For labeling principles, the labels for sentiment are 'Positive', 'Negative', and 'Neither', as used in S. Mohammad et al. (2016). The labels for stance are 'Support', 'Against', and 'Neither'. If we cannot explicitly or implicitly determine the author's stance from the text, the stance does not necessarily correspond to 'Neutral' (Küçük & Can, 2020). 'Neither' is preferred over 'Neutral' to cover cases other than 'Support' and 'Against' (Küçük & Can, 2020).

Table 1 shows the distribution of each class in the train and test sets for sentiment and stance. We can see the data is imbalanced, with "Positive" and "Support" as the minority classes. Tables 2 and 3 show the distribution of sentiment and corresponding stance pairs in the training and test sets, respectively. Table 2 presents the distribution for the training set, while Table 3 presents the distribution for the test set. These tables illustrate that sentiment is not necessarily equivalent to stance. As we can see, positive sentiment can indicate different stances, and the same applies to other sentiments.

Table 1: Sentiment and Stance Labeled Data in both Train and Test Sets

		Positive	Negative	Neither	Total
Sentiment	train	48	262	98	408
	test	38	245	124	407
		Support	Against	Neither	Total
Stance	train	82	186	140	408
	test	76	191	140	407

Table 2: Counts of Sentiment and Corresponding Stance Pairs in the Train Set

	Positive	Negative	Neither	Total
Support	22	53	7	82
Against	13	158	15	186
Neither	13	51	76	140
Total	48	262	98	408

Table 3: Counts of Sentiment and Corresponding Stance Pairs in the Test Set

	Positive	Negative	Neither	Total
Support	25	38	13	76
Against	5	170	16	191
Neither	8	37	95	140
Total	38	245	124	407

4.2.2 *Data Augmentation*

We can see that our data has an imbalance problem and a small sample size. To address these issues, we use Easy Data Augmentation (EDA), which includes the following four operations:

- Synonym replacement: Select n non-stopwords at random from a sentence and substitute each with a randomly chosen synonym.

- Random insertion: Identify a random non-stopwords in the sentence, find its synonym, and insert it at a random spot within the sentence. Repeat this process *n* times.

- Random swap: Select two words from the sentence with randomness and exchange their places. Repeat this process *n* times.

- Random deletion: Randomly eliminate each word in the comment with probability *p*.

As demonstrated, synonyms are a crucial resource for employing this method. Currently, only English and Chinese versions are available. For Dutch synonyms, we used OpenDutchWordnet (Postma et al., 2016), a Dutch thesaurus containing 117,914 synonym sets, with 51,588 sets including at least one Dutch synonym. We integrated these synonym sets into *eda.py*, the main file of the EDA technique, by modifying the NLTK (Natural Language Toolkit) English synonyms to Dutch ones within the functions.

Since the number of augmentations applied to each class can be different. For sentiment classes, the augmentation times are set to 9 for 'Positive', 1 for 'Negative', and 4 for 'Neither'. For stance labels, the augmentation times are 6 for 'Support', 2 for 'Against', and 3 for 'Neither'. These numbers are chosen specifically to address data imbalance. Each operation's implementation percentage is set at 0.1. For instance, in the original sentence, 10% of the words will be replaced with synonyms (alpha_sr = 0.1).

4.2.3 *Data Preprocessing*

Crawled data often contains noise such as URLs, emojis, and other non-characters. According to Küçük and Can (2020), common preprocessing tasks include removing specific tokens and characters, normalization, case conversion, and tokenization. These processes vary depending on the feature extraction methods used.

For TF-IDF, preprocessing involves removing URLs and non-letter characters, converting text to lowercase, and removing stopwords. For pre-trained embeddings, the steps are similar, except that stopwords are retained because they carry important contextual information and are included in the corpora used to train word2vec and FastText. We avoid stemming or lemmatizing to preserve the original semantic meaning. Finally, we map the pre-trained embeddings to our data and calculate the average word vectors as sentence features.

4.3 *Method*

4.3.1 *Feature Extraction*

TF-IDF TF-IDF (Term Frequency Inverse Document Frequency) measures the importance of words in a document by considering their frequency within the document and their rarity across the entire document set (Ramos et al., 2003). The formula is:

$$tfidf(w, d) = tf \cdot \log\left(\frac{N+1}{N_w+1}\right) + 1$$

In this formula, tf is the term frequency, representing how often word w appears in document d. The rest part of the formula is for calculating the Inverse Document Frequency (IDF), where N is the total number of documents, and N_w is the number of documents containing the specific word w. It evaluates the distinctiveness of word w in the whole document d. Higher IDF values indicate more distinctive words, while lower values indicate common words (Ramos et al., 2003).

WORD2VEC The limitation of TF-IDF is that it fails to capture the semantic meaning of each word and could cause high dimensional and sparse data that contains a lot of zeros, which is called the curse of dimensionality (Bengio et al., 2000).

Word2vec is a revolutionized technique proposed by Mikolov et al. (2013) which could solve the mentioned problems above. It could capture syntactic and semantic relationships of words and avoid sparsity at the same time. There are two engines of word2vec: Continuous Skip-gram architecture and Continuous Bag-of-Words (CBOW) architecture. Skiggram predicts the context from the target.

In our research, we utilize pre-trained word2vec embeddings specifically for the Dutch language, which are trained using the Continuous Skip-gram algorithm and have 100 dimensions. The resource could be downloaded here: http://vectors.nlpl.eu/repository/.

FASTTEXT FastText, introduced by Facebook (Bojanowski et al., 2017), extends the word2vec model by incorporating subword information, specifically character n-grams. This makes FastText well-suited for morphologically rich languages, as it can better handle inflections and compound words. Unlike word2vec, which struggles with rare words and neologisms, FastText's subword approach allows for better representations of uncommon terms by analyzing their internal structure. This is crucial for our Reddit-derived dataset, which likely includes slang and varied morphological forms not present in traditional Dutch corpora. The FastText embeddings used in this study were trained on data from Wikipedia and Common Crawl, as described by Grave et al. (2018), and are available at https://FastText.cc/docs/en/crawl-vectors.html.

4.3.2 *Machine Learning Models*

LOGISTIC REGRESSION For our three-class classification task, we use multinomial Logistic Regression (LR), an extension of binary LR for predicting outcomes with more than two classes. Multinomial LR is a method that analyzes input data to predict which category an item belongs to and provides the probability for each category (Bishop & Nasrabadi, 2006). Unlike tree-based models, LR is a linear model that assumes a linear relationship between the input features and the log-odds of the output classes.

RANDOM FOREST Random Forest (RF) is an ensemble learning method that builds multiple decision trees and merges them to get a more accurate and stable prediction. RF utilizes a modified version of Bagging, where each tree is built independently using a random sample of data points, and nodes are split based on a randomly selected subset of features (Hastie et al., 2009). This random feature selection prevents overfitting and adds diversity to the model (Akar & Güngör, 2012). RF is known for its robustness and effectiveness with high-dimensional data and small sample sizes (Biau & Scornet, 2016), but it can be computationally intensive due to the large number of trees.

XGBOOST The disadvantages of Random Forest include being computationally and memory-intensive, and its decision trees might make the same errors since they are trained independently. XGBoost, which stands for eXtreme Gradient Boosting, addresses these issues by using a gradient boosting framework that builds decision trees sequentially. Each subsequent tree corrects the errors of its predecessors, improving model performance.

XGBoost's success is largely due to its scalability, as noted by T. Chen and Guestrin (2016), who developed the algorithm. It efficiently processes large datasets without significantly increasing computational time or memory usage. This is achieved through innovations like the weighted quantile sketch and out-of-core computing (T. Chen & Guestrin, 2016). These features make XGBoost well-suited for our tasks, especially given the high-dimensional vectors from pre-trained embeddings and the long sentences from Reddit data. Additionally, our EDA technique will expand the dataset, making XGBoost's efficiency and scalability ideal for handling these complexities.

5 EXPERIMENT SETUP

5.1 *Model Training and Hyperparameter tuning*

5.1.1 *Model Training with Nested Cross Validation*

We employed nested cross-validation to train the models. In the outer loop, we performed a stratified shuffle split of the data into five folds to maintain the original label proportions. We conducted experiments with and without Easy Data Augmentation (EDA) to determine whether EDA enhances model performance. When training with EDA, we applied EDA exclusively to the training portions of the folds to enhance model robustness, while the test parts of the outer loop remained unchanged. Each iteration will generate new augmented data. For the inner loop, we performed a stratified shuffle split of the training data into three folds for hyperparameter tuning. The macro F1 score was chosen as the evaluation metric for both loops due to the imbalanced nature of our sentiment and stance data.

Next, we selected the top three models for sentiment analysis and stance detection based on the average macro F1 scores from the nested cross-validation, separately. These top-performing models were then applied to the test set for out-of-sample evaluation to assess their generalization ability. Subsequently, we performed error analysis on the models that achieved the best results on the test data. Finally, to explore the impact of additional features on model performance, we focused on the top three models trained without EDA in both sentiment analysis and stance detection. We investigated whether incorporating these additional features could further improve the performance of these top-performing models. The entire process is illustrated in Figure 1.

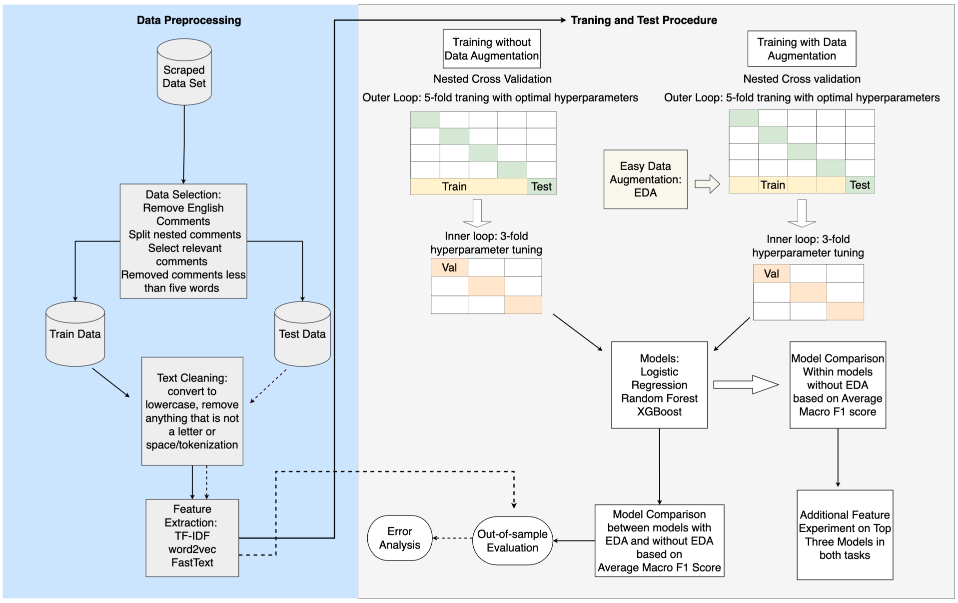

Figure 1: Experiment Flow Chart: Sentiment Analysis and Stance Detection on PVV-related Reddit Comments using Machine Learning with TF-IDF, word2vec, FastText, EDA and Additional Feature in a Nested Cross Validation Framework

5.1.2 *Hyperparameter Tuning*

TUNING LR The hyperparameters for Logistic Regression are *Inverse of regularization strength* (C), *solver*, and *max_iter* (see Table 4). The parameter C balances underfitting and overfitting; smaller values indicate stronger regularization. We use logspace(-4, 4, 20) to generate 20 values on a logarithmic scale between 10^{-4} and 10^4. For our multi-classification task, we choose the solvers newton-cg, lbfgs, and saga, with lbfgs being particularly effective for small sample sizes. The *max_iter* parameter defines the maximum number of iterations allowed for convergence, with values of 20,000, 30,000, and 40,000 to ensure sufficient iterations.

Table 4: Hyperparameter Space for Logistic Regression

Hyperparameter	Values
Inverse of regularization strength (C)	$\text{logspace}(-4, 4, 20)$
solver	{newton-cg, lbfgs, saga}
max_iter	{20000, 30000, 40000}

TUNING RF The hyperparameters for Random Forest include *n_estimators*, *criterion*, *max_depth*, *min_samples_split*, *min_samples_leaf*, *max_features*, and *bootstrap*. *n_estimators* and *max_depth* are crucial, as they determine the number of decision trees and the depth of each tree. We set *n_estimators* between 10 and 100, and *max_depth* from 2 to 50. More trees and greater depth capture complex patterns but may overfit the training data. The *criterion* choices are gini and entropy; gini is typically faster, while entropy is more information-theoretic. The complete hyperparameter range is shown in Table 5. These values aim to fit the overall data better while avoiding overfitting.

Table 5: Hyperparameter Space for Random Forest

Hyperparameter	Values
n_estimators	10 to 100
criterion	gini, entropy
max_depth	2 to 50
min_samples_split	2 to 10
min_samples_leaf	1 to 10
max_features	sqrt, log2, None
bootstrap	True, False

TUNING XGBOOST We set six hyperparameters for XGBoost, as shown in Table 6. The *learning_rate* is set between 0.01 and 0.3. *max_depth*, *subsample*, and *colsample_bytree* help prevent overfitting. *max_depth* is limited to a maximum of 10. *subsample* ranges from 0.5 to 1.0, defining the fraction of samples used for fitting individual base learners. *colsample_bytree* is set between 0.5 and 1.0, determining the fraction of features randomly sampled for each tree. The *lambda* and *alpha* parameters are for L1 and L2 regularization, respectively. According to P. Zhang et al. (2022), higher regularization values can lead to underfitting, so we set them between 0 and 1.0.

Table 6: Hyperparameter Space for XGBoost

Hyperparameter	Values
learning_rate	0.01 to 0.3
max_depth	3 to 11
subsample	0.5 to 1.0
colsample_bytree	0.5 to 1.0
lambda	0 to 1.0
alpha	0 to 1.0

5.2 *Evaluation Metrics*

5.2.1 *Krippendorff Alpha*

Krippendorff's alpha (α) measures inter-rater reliability for nominal data among multiple raters, making it suitable for our categorical data annotated by two raters. The formula is:

$$\alpha = \frac{P_o - P_e}{1 - P_e}$$

where P_o is the observed agreement, and P_e is the expected agreement by chance. Alpha ranges from -1 to 1, with -1 indicating complete disagreement, 0 indicating no agreement beyond chance, and 1 indicating perfect agreement. This method helps determine the ground truth for both train and test sets in sentiment and stance. Table 7 shows an agreement of 36% and 41% for sentiment, and 57.08% and 57.28% for stance. However, a value of at least 0.667 is needed for tentative conclusions.

$$\alpha = \frac{P_o - P_e}{1 - P_e} \tag{1}$$

Table 7: Krippendorff Alpha Results for Sentiment and Stance

	Train Set	Test Set
Sentiment	36.10%	41.25%
Stance	57.08%	57.28%

5.2.2 *Macro Average F1 score*

Since the data is imbalanced, we use the macro average F1 score to treat each class equally . The macro average F1 score is calculated by taking the average of the F1 scores of all classes.

$$F1_{\text{macro}} = \frac{1}{N} \sum_{i=1}^{N} F1_i \tag{2}$$

6 RESULTS

6.1 *Model Performance with Different Embeddings Without EDA*

The table 8 reports the average macro F1 scores of different models using TF-IDF, word2vec, and FastText for both sentiment analysis and stance detection tasks. The highest F1 scores are highlighted in bold.

The ground truth for sentiment labels in the training set is 36.10%, and for stance labels, it is 57.08%. In sentiment analysis, most models surpass the ground truth, except for RF with TF-IDF and RF with FastText. Notably, RF achieves the highest average macro F1 score at 41.84%. In stance detection, no model outperforms the ground truth, indicating the challenge of this task. The best-performing model in stance detection is XGBoost with word2vec, achieving an average macro F1 score of 49.34%.

Word2vec embeddings consistently demonstrate superior performance compared to TF-IDF and FastText across both tasks. This suggests that word2vec is more effective at capturing the nuances of the text. For instance, in sentiment analysis, word2vec with RF achieves 41.84%, significantly outperforming TF-IDF with RF at 34.74%. Similarly, in stance detection, word2vec achieves the best F1 scores across most models, with LR reaching 49.26% and XGBoost achieving 49.34%.

Comparing FastText with the TF-IDF reveals several key observations. In sentiment analysis, FastText generally achieves competitive performance but does not consistently surpass TF-IDF. For example, FastText's performance with LR at 37.29% remains below TF-IDF's 39.27%. A similar trend is observed in stance detection, where FastText, despite being competitive, often falls short of the TF-IDF scores, such as achieving 39.14% with XGBoost compared to TF-IDF's 41.89%.

Table 8: Average macro F1 scores of models with different embeddings in sentiment analysis and stance detection using nested cross-validation without EDA

Embedding	Sentiment Analysis			Stance Detection		
	LR	RF	XGBoost	LR	RF	XGBoost
TF-IDF	39.27%	34.74%	36.57%	45.60%	39.99%	41.89%
word2vec	**40.51%**	**41.84%**	37.34%	**49.26%**	**48.29%**	**49.34%**
FastText	37.29%	35.31%	**39.63%**	44.23%	42.93%	39.14%

6.2 *The Impact of EDA on Model Performance*

To illustrate the impact of EDA, we employ boxplots to compare the median macro F1 score and data dispersion. Figures 2 and 3 depict the effects of EDA on sentiment and stance classification across the same set of model-embedding combinations. The x-axis represents the nine model-embedding combinations, while the y-axis shows macro F1 scores from nested cross-validation. The blue line indicates models without EDA, and the orange line indicates models with EDA. A t-test was conducted on the performance of each model before and after using EDA, and the results showed no significant change.

In sentiment analysis, EDA generally enhances the performance of several models and embeddings, as evidenced by 7 out of 9 model-embedding combinations displaying a higher median macro F1 score with EDA. Notably, XGBoost with word2vec achieves the highest median F1 score with EDA at 46.30%. However, EDA does not uniformly benefit all combinations. For instance, the RF model with word2vec embedding performs better without EDA, achieving a median macro F1 score of 45.47% without EDA compared to 38.33% with EDA, which represents the largest performance gap among the evaluated models.

In stance analysis, 4 out of 9 models show improved performance with EDA, achieving higher median macro F1 scores. While LR with TF-IDF and XGBoost with FastText benefit from EDA, with the former achieving the highest median macro F1 score of 50.19%, other combinations such as XGBoost with word2vec perform better without it.

From the boxplots, we can also observe the variability in the performance of different models. In both tasks, the introduction of EDA increases variance and instability across folds in some configurations. For instance, in sentiment analysis, XGBoost with FastText exhibits a more compact distribution without EDA compared to its performance with EDA. In stance detection, LR with word2vec and EDA displays higher variance compared to its performance without EDA. The increased variance suggests that while some folds achieve high scores, others perform significantly worse.

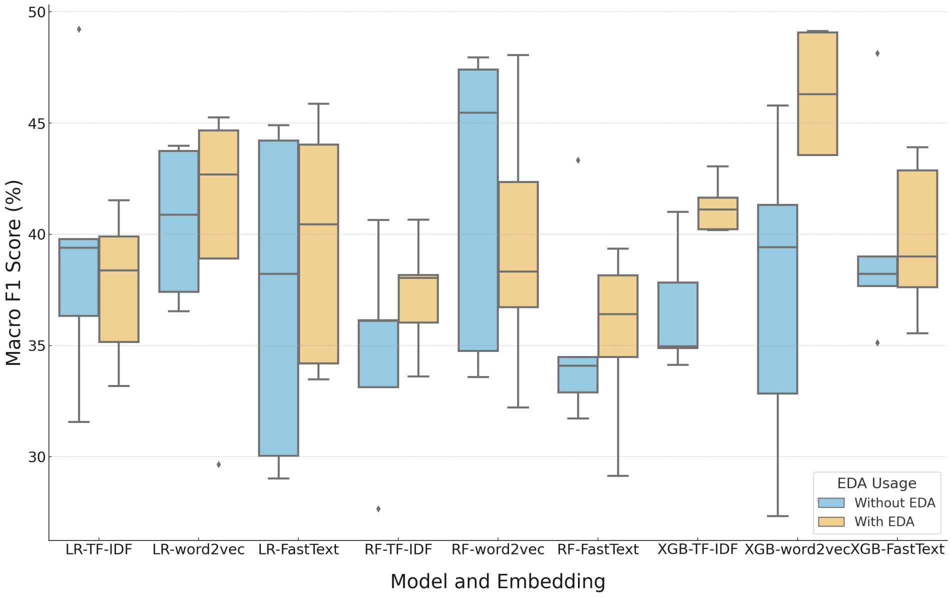

Figure 2: Comparison of Model Performance in Sentiment Analysis Using Boxplots: Macro F1 Scores With and Without EDA After Nested Cross-Validation

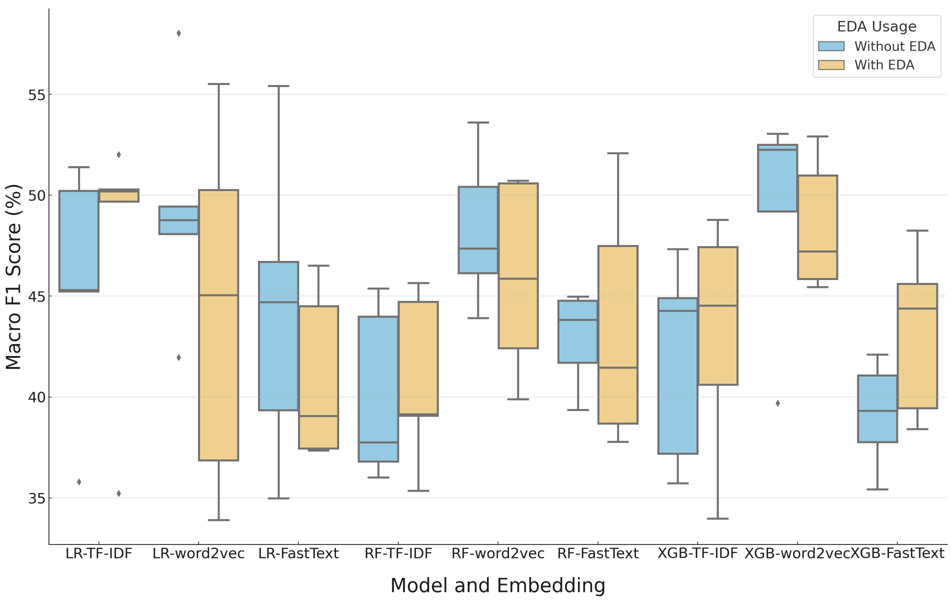

Figure 3: Comparison of Model Performance in Stance Detection Using Boxplots: Macro F1 Scores With and Without EDA After Nested Cross-Validation

6.3 *Out-of-sample Evaluation*

The aim of this section is to evaluate the generalization ability of the models. We selected the top three models for both classification tasks. The ranking tables of all model performances for both tasks are provided in Appendix C. The optimal hyperparameter sets for each model are provided in Table 9, and the complete optimal hyperparameter sets for all models in both tasks are detailed in Appendix B. Using these optimal hyperparameters, the models were re-trained on the entire training dataset and subsequently evaluated on a separate test dataset to assess their generalization ability. Table 10 presents the results, showing the macro F1 scores for both the training and test sets to facilitate comparison.

Table 9: Optimal Hyperparameter Sets for the Top Three Models in Sentiment Analysis and Stance Detection

Type	Model Combination	Best Hyperparameter Set
Sentiment	XGBoost + word2vec with EDA	subsample: 0.5, max_depth: 6, learning_rate: 0.3, lambda: 0.7, colsample_bytree: 0.5, alpha: 0.1
	RF + word2vec without EDA	n_estimators: 83, min_samples_split: 7, min_samples_leaf: 10, max_features: sqrt, max_depth: 37, criterion: gini, bootstrap: False
	XGBoost + TF-IDF with EDA	subsample: 0.6, max_depth: 10, learning_rate: 0.3, lambda: 0.9, colsample_bytree: 0.8, alpha: 0.2
Stance	XGBoost + word2vec without EDA	subsample: 0.6, max_depth: 5, learning_rate: 0.13, lambda: 0.1, colsample_bytree: 0.8, alpha: 0.2
	LR + word2vec without EDA	solver: saga, max_iter: 40000, C: 29.763514416313132
	XGBoost + word2vec with EDA	subsample: 0.6, max_depth: 8, learning_rate: 0.25, lambda: 0.4, colsample_bytree: 0.8, alpha: 0.1

Table 10: Comparison of Generalization Ability of Top Three Models in Sentiment Analysis and Stance Detection Based on Average Macro F1 Score

Type	Model	Embedding	Avg. F1 Macro (Train)	F1 Macro (Test)	EDA	Drop
Sentiment	XGBoost	word2vec	**43.23%**	37.63%	with	**5.60%**
	RF	word2vec	41.84%	**46.48%**	without	-4.64%
	XGBoost	TF-IDF	41.25%	36.37%	with	4.88%
Stance	XGBoost	word2vec	**49.34%**	**46.49%**	without	2.85%
	LR	word2vec	49.26%	42.93%	without	**6.33%**
	XGBoost	word2vec	48.48%	45.06%	with	3.42%

For sentiment analysis, the RF model with word2vec embeddings without EDA demonstrates the best generalization ability, as indicated by a negative drop (-4.64%), meaning it performs better on the test set than on the training set. Conversely, the XGBoost model with word2vec embeddings and EDA, which achieved the highest performance during training with an F1 score of 43.23%, experienced a significant decline to 37.63% on the test set, representing the largest decrease of 5.6%. The XGBoost model with TF-IDF with EDA exhibited the performance drop of 4.88%.

For stance detection, all three models showed a performance drop from training to testing. The XGBoost model with word2vec without EDA achieved the highest F1 score on the test set at 46.49%, with the smallest drop of 2.85%, indicating better generalization ability compared to the other models. Its counterpart with EDA experienced a slightly larger drop of 3.42%, from 48.48% to 45.06%. The LR model with word2vec without EDA showed the largest drop of 6.33%, from 49.26% to 42.93%.

Overall, models without EDA tend to exhibit better generalization ability, as they show smaller performance drops from training to testing or even improve their performance on the test set. In contrast, models with EDA demonstrate performance drops from training to testing in all cases. Additionally, the best-performing models in both sentiment analysis and stance detection on the test set are those without EDA.

6.4 *Error Analysis*

6.4.1 *Error Breakdown*

According to section 6.3, the best-performing model for sentiment analysis is RF with word2vec without EDA, while for stance detection, it is XGBoost with word2vec without EDA.

Table 11: Classification Report for the RF+word2vec Model without EDA in Sentiment Analysis

Target Label	Precision	Recall	F1-score	Support
0	26.32%	13.16%	17.54%	38
1	67.36%	79.18%	72.80%	245
2	55.00%	44.35%	49.11%	124
accuracy			62.42%	407
macro avg	49.56%	45.57%	46.48%	407
weighted avg	59.76%	62.41%	60.42%	407

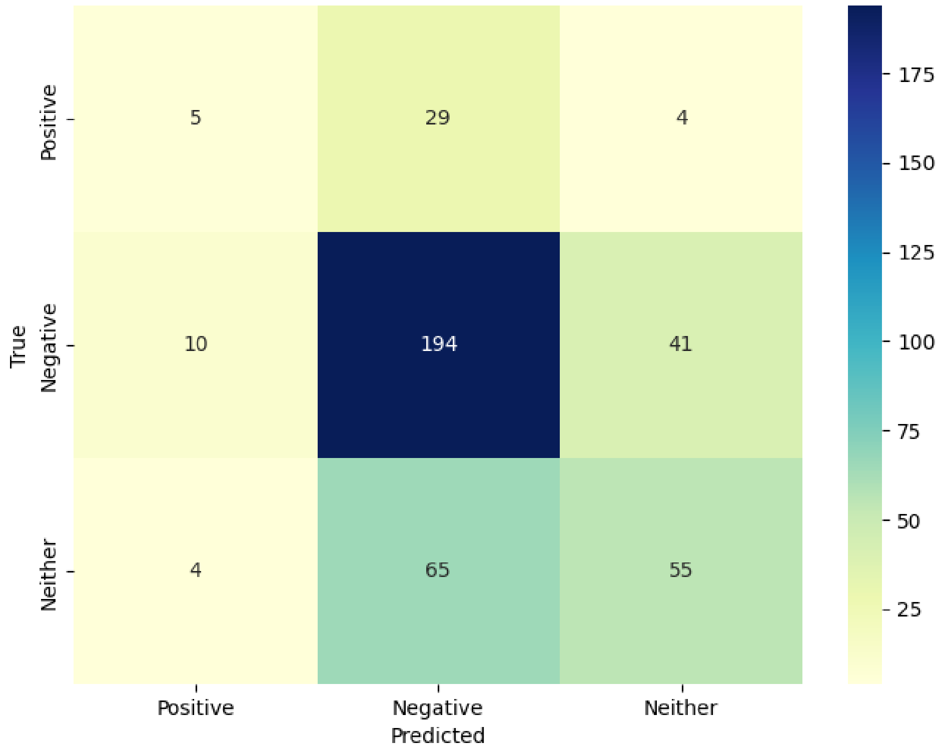

Figure 4: Confusion Matrix for the RF+word2vec Model without EDA in Sentiment Snalysis

Table 11 and Figure 4 present the classification report and confusion matrix of the best-performing model for sentiment analysis. The model struggles to accurately predict class 0 (Positive), with an F1-score of 17.54%. It performs better for class 1 (Negative) and class 2 (Neither), with F1-scores of 72.80% and 49.11%, respectively. For class 0 (Positive), out of 38 samples, only 5 are correctly classified as 'Positive,' while 29 are misclassified as 'Negative' (class 1) and 4 as 'Neither' (class 2). This indicates that the model often confuses 'Positive' with 'Negative.' For class 1 (Negative), out of 245 samples, 194 are correctly classified. However, 41 samples are misclassified as 'Neither,' and 10 as 'Positive.' The 'Neither' class (class 2) also poses challenges for the model. Out of 124 samples, 55 are correctly classified, but 65 are misclassified as 'Negative,' and 4 as 'Positive.' This suggests that the model often confuses 'Neither' with 'Negative.'"

Table 12: Classification Report for the XGBoost+word2vec Model without EDA in Stance Detection

Target Label	Precision	Recall	F1-score	Support
0	40.00%	18.42%	25.23%	76
1	54.47%	67.02%	60.09%	191
2	54.74%	53.57%	54.15%	140
accuracy			53.32%	407
macro avg	49.74%	46.34%	46.49%	407
weighted avg	51.86%	53.32%	51.54%	407

Table 12 and Figure 5 present the classification report and confusion matrix of the XGBoost model with word2vec without EDA for stance detection. The model struggles to accurately predict class 0 (Support), with an F1 score of 25.23%. It performs better for class 1 (Against) and class 2 (Neither), with F1 scores of 60.09% and 54.15%, respectively. For the Support class, 14 out of 76 samples are predicted correctly, 47 are misclassified as Against, and 15 are misclassified as Neither. This indicates that the model often confuses "Support" with "Against". For the Against class, 128 out of 191 samples are correctly predicted, 16 are predicted as Support, and 47 are predicted as Neither. For the Neither class, 75 out of 140 samples are predicted correctly, 5 are predicted as Support, and 60 are predicted as Against. This indicates that the model often confuses "Neither" with "Against".

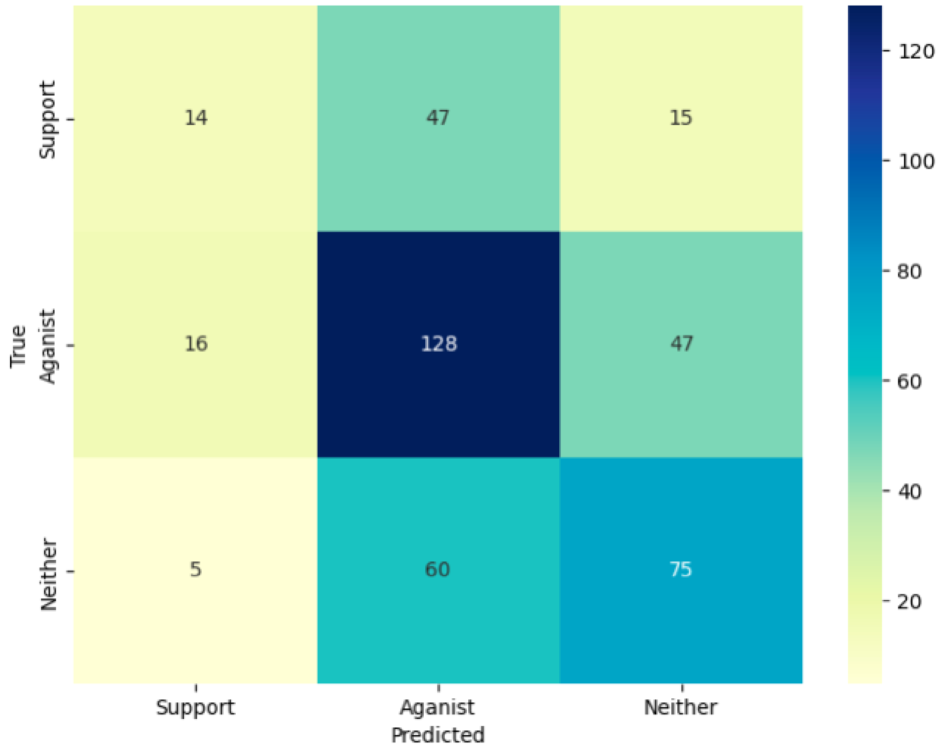

Figure 5: Confusion Matrix for the XGBoost+word2vec Model without EDA in Stance Detection

6.4.2 *Bias Detection*

To further analyze the model for potential biases, we introduce bias detection by examining how the model performs across different groups. Specifically, for the sentiment analysis model, we group the data by stance labels and compare the macro F1 scores. For stance detection, we group the data by sentiment labels. Table 14 shows the results, highlighting potential biases and the model's overall performance across different classes.

The best sentiment analysis model is RF with word2vec embedding without EDA, achieving a macro F1 score of 46.48%. Table 13 reveals that the model performs similarly in the Support and Against groups, with macro F1 scores of 45.31% and 45.52%, respectively. However, the macro F1 score for the Neither group is 33.87%, 21% lower than the Support and Against Groups, indicating huge bias.

The best stance detection model is XGBoost with word2vec without EDA, also achieving a macro F1 score of 46.49%. Table 13 shows the model performs similarly in the Negative and Neither groups, with macro F1 scores of 43.91% and 42.03%, respectively. However, the F1 macro for the

Positive group is 20.08%, more than 20% lower than the Negative and Neither groups, indicating bias against the Positive group.

Table 13: Bias Detection of the Best Generalization Models in Sentiment Analysis and Stance Detection

Model	Group	F1 Macro
Sentiment	Support	45.32%
	Against	45.52%
	Neither	33.87%
Stance	Positive	20.08%
	Negative	43.91%
	Neither	42.03%

6.5 *The Impact of Additional Features*

Tables 14 and 15 present performance comparisons for the top three sentiment analysis and stance detection models without EDA, respectively, before and after adding additional features. The tables include average macro F1 scores and standard deviations. A t-test was performed to determine the statistical significance of the observed improvements, showing that although the additional features can improve performance, the improvements are not statistically significant. The better results within pairs are highlighted in bold.

Table 14 shows the performance of the top three sentiment analysis models without EDA before and after adding stance labels as additional features. The models consistently show improved average F1 macro scores when stance labels are added: for LR, the score increased from 40.51% to 47.92%; for RF, the score increased from 41.84% to 48.54%; and for XGBoost, the score increased from 39.63% to 44.69%. The standard deviation generally decreases, indicating reduced variability across folds.

Similarly, Table 15 displays the performance of the top three stance detection models without EDA before and after adding sentiment labels as additional features. The addition of sentiment labels improves the average F1 macro scores: for LR, the score increased from 49.26% to 57.11%; for RF, the score increased from 48.29% to 55.88%; and for XGBoost, the score increased from 49.34% to 52.95%. The standard deviation decreases in most cases, suggesting increased stability: for LR, it decreased from 5.14%

to 3.93%; for RF, it decreased from 3.39% to 2.63%; and for XGBoost, it decreased from 5.01% to 2.06%.

These improvements in F1 scores after adding additional features indicate that these features provide valuable information that enhances model performance. The reduction in standard deviation suggests that the models become more stable and less sensitive to variations in the training data.

Table 14: Performance Comparison of the Top Three Sentiment Analysis Models without EDA with Stance Label as an Additional Feature

Model	Feature	Train	
		Avg. F1 Macro	Std. Dev.
LR	word2vec	40.51%	3.10%
	word2vec + stance	**47.92%**	4.46%
RF	word2vec	41.84%	6.32%
	word2vec + stance	**48.54%**	3.99%
XGBoost	FastText	39.63%	4.44%
	FastText + stance	**44.69%**	2.74%

Table 15: Performance Comparison of the Top Three Stance Detection Models without EDA with Sentiment Label as an Additional Feature

Model	Feature	Train	
		Avg. F1 Macro	Std. Dev.
LR	word2vec	49.26%	5.14%
	word2vec + sentiment	**57.11%**	3.93%
RF	word2vec	48.29%	3.39%
	word2vec + sentiment	**55.88%**	2.63%
XGBoost	word2vec	49.34%	5.01%
	word2vec + sentiment	**52.95%**	2.06%

7 DISCUSSION

This study predicts sentiment and stance at the same time using machine learning on PVV-related Reddit comments during the 2023 Dutch general election, and explores the relationship between sentiment and stance. To address this, we proposed several sub-questions, examining the performance of Logistic Regression, Random Forest, and XGBoost models using different embeddings (TF-IDF, word2vec, and FastText). Additionally, we explored the impact of Easy Data Augmentation (EDA) on model performance and generalization ability, investigating whether models with EDA demonstrate better generalization ability compared to those without EDA. Furthermore, we conducted bias detection to determine if the models perform consistently across different groups. We investigated whether incorporating sentiment and stance labels as additional features could enhance model performance. This section aims to evaluate and interpret the previously presented results, providing a comprehensive understanding of the entire study.

7.1 *Result Discussion*

7.2 *Discussion on Model Performance with Different Embeddings Without EDA*

Overall, the models' performance is suboptimal, with all falling far below 100%. For sentiment analysis, the highest accuracy achieved during the training phase was 41.84%, which is comparable to the best F1 score of 0.41 achieved by TextBlob as reported by Korbee (2021) in 2021 Dutch general election. This indicates that machine learning models cannot surpass the performance of lexicon-based approaches. The overall lower performance might be due to the small sample size and imbalanced data used for training, which hinders the models' ability to capture the essential differences between each class. It is notable that the same models and embeddings perform better in stance detection overall than in sentiment analysis. This might be attributed to the higher inter-rater reliability of stance detection.

The results indicate that feature extraction has an impact on the performance of the models. Word2vec outperforms FastText and TF-IDF in both sentiment analysis and stance detection tasks. This finding aligns with the work of Dang et al. (2020), who demonstrated that word2vec outperforms TF-IDF.

Although FastText incorporates sub-word information, which theoretically should offer an advantage in capturing morphological details and handling rare or out-of-vocabulary words (da Costa et al., 2023), this advantage is not apparent in our tasks. While FastText provides competitive performance and occasionally exceeds TF-IDF, particularly when used with more complex models like XGBoost, it does not consistently outperform the TF-IDF across all models and tasks. This may be attributed to the fact that FastText embeddings are not specifically trained on political texts, but rather on general corpora such as Wikipedia and web crawled data (Grave et al., 2018).

7.3 *Discussion about the EDA performance*

The impact of EDA is inconsistent; it does not always enhance model performance and can sometimes decrease it, introducing variability across folds. The marginal improvements align with results by J. Wei and Zou (2019) and Wirawan, Cahyono, et al. (2023).

Several factors may contribute to these outcomes. First, the inherently random nature of the four operations—synonym replacement, random insertion, random swap, and random deletion—plays a significant role. When applying EDA within nested cross-validation, the augmented sentences generated by EDA can vary with each iteration of the outer cross-validation training partitions, potentially explaining the observed variability in performance across folds.

Second, according to Kumar et al. (2020), EDA could potentially alter sentence semantics. This may lead to discrepancies between the sentiment or stance of the augmented data and the original labels, thereby causing model confusion and resulting in unstable performance.

Third, the proportion of each operation in the EDA technique can also influence the performance outcomes (J. Wei & Zou, 2019). In our research, we tested a 0.1 proportion and did not explore other values.

Finally, when the number of instances for less frequent labels is larger, there may not be enough synonyms to replace or insert, nor sufficient words to randomly swap or delete. This can lead to duplicates in the data and diminish the quality of the augmentation.

7.4 *Discussion about the generalization ability*

Based on the results, most models without EDA and all models with EDA exhibit varying degrees of performance decline on the test set compared to the training set. However, overall, the generalization ability of models without EDA is superior to that of models with EDA. The performance decline in models without EDA may be attributed to insufficient training data, whereas the decline in models with EDA may result from the introduction of noise during the training phase by EDA, thereby affecting the model's judgment and its performance on the test data.

7.5 *Discussion about the error analysis*

For sentiment analysis, the model performs poorly on the Positive class, frequently confusing "Positive" with "Negative" and "Neither" with "Negative." In stance detection, the model underperforms on the Support class, often misclassifying "Support" as "Against" and "Neither" as "Against." This low performance may be attributed to Positive and Support being minority classes in the training data, especially after nested cross-validation, where the data in each fold is limited, resulting in insufficient data for the model to learn effectively.

The bias in sentiment prediction indicates that the sentiment analysis model struggles to accurately determine sentiment in cases where the stance is ambiguous, unlike the more distinct sentiments of the Support or Against groups, which show relatively clearer sentiment distinctions.

The stance detection model struggles to distinguish stances within the positive sentiment group, suggesting that positive sentiment may imply a complex stance towards the target PVV. In contrast, the model finds it relatively easier to determine the stance within the negative sentiment and neither sentiment groups. This may be because the majority of instances in the negative sentiment group are against to PVV, and most instances in the neither sentiment group have ambiguous stances.

Table 2 illustrates these findings. Compared to other sentiment groups, the stance distribution within the positive sentiment group is relatively less imbalanced. In the negative sentiment group, there are 158 instances corresponding to an against stance, while fewer instances correspond to support (53) and neither (51) stances. For the neither sentiment group, 75 instances correspond to an neither stance, with fewer instances corresponding to support (7) and against (15) stances. This indicates that the model had inherent bias during training, which persisted during testing.

7.6 *Discussion about the additional feature*

Adding the additional feature could improve model performance, which constitutes the novel aspect of our research. In the literature, S. M. Mohammad et al. (2017) and Buytaert (2018) incorporate sentiment lexicons as additional features. Küçük and Arıcı (2024) introduces the sentiment feature in stance detection and the stance feature in sentiment analysis. However, the author does not assess whether the additional feature could enhance model performance. S. M. Mohammad et al. (2017) evaluates the impact of sentiment lexicons, demonstrating that sentiment analysis benefits from their inclusion, whereas stance analysis does not. Our research, however, shows that stance analysis benefits from the additional sentiment feature. This operation involves incorporating data that is potentially unseen as a feature, which may lead to data leakage and consequently result in misleading performance metrics.

However, these consistent improvements still highlight the close connection between sentiment and stance, emphasizing their complementary roles in both sentiment analysis and stance detection. Although sentiment and stance are not entirely the same, there is a relationship between them. Sentiment labels can provide additional semantic information for stance analysis, enabling the model to better comprehend the stance of the text. Similarly, stance labels can offer contextual clues for sentiment analysis, aiding the model in more accurately identifying sentiments.

7.7 *Broader Societal Impact*

This study finds that machine learning's ability to predict sentiment and stance in PVV-related Reddit comments is limited. Word2vec outperforms TF-IDF and FastText. EDA, as a data augmentation technique, is not stable. Although these findings need further exploration due to the low inter-rater reliability, this study still demonstrates that sentiment and stance, while not entirely the same, have complementary effects on the performance of machine learning models. This underscores the importance of considering both sentiment and stance when analyzing political texts, rather than focusing solely on sentiment.

The predictive models developed in this research can inspire the creation of an enhanced online platform for real-time monitoring of public sentiment and stance. Such a platform could aid candidates in understanding voters' sentiment and stance, as well as their changes at different stages, thereby allowing candidates to adjust their campaign strategies accordingly. For instance, following a significant debate, sentiment analysis and stance detection can reveal immediate voter reactions, enabling candidates to promptly adjust their strategies. Additionally, stance analysis can be extended to identify voters' stances on specific issues, such as anti-Muslim sentiments, environmental protection, or proposals from other candidates. This publicly available information can also help voters remain vigilant when obtaining information, avoiding being misled by biased reports from media with a singular stance.

7.8 *Limitations*

A major limitation of our thesis is the low inter-rater reliability between the two annotators, despite following the same labeling guidelines. As mentioned in Section 5.2.1, a minimum agreement of 0.67 is required to draw tentative conclusions. This low reliability could be attributed to several factors: the annotators may lack professional training, political text may be inherently difficult to categorize, or the labeling guidelines may be insufficiently clear. For sentiment, annotators could encounter situations that are challenging to label as positive or negative, such as sarcasm and requests.

Another limitation is the small sample size of our labeled data, which may not provide sufficient training for the models. Additionally, although we used data augmentation techniques, we did not explore the hyperparameters of these techniques, such as the percentage of each operation, which could have impacted the results.

7.9 *Future Direction*

In future work, we could explore deep learning methods with different word embeddings and fine-tuning transformer models to assess their impact on model performance. Additionally, utilizing pre-trained embeddings for comparison could provide valuable insights, as these embeddings offer contextual information that enhances the understanding of text data. Further exploration of alternative data augmentation methods or adjustments to the percentage of each augmentation operation could also help observe variations in the results.

8 CONCLUSION

In this section, we present the conclusions regarding our research questions based on our experiments. However, these conclusions require further exploration due to the inter-rater reliability of our ground truth being below 0.67.

RQ1 *How do Logistic Regression, Random Forest, and XGBoost with TF-IDF, word2vec, and FastText perform in sentiment analysis and stance detection compared to the ground truth?*

The performance of Logistic Regression, Random Forest, and XGBoost models varies depending on the embedding method used. Word2vec embeddings generally outperform TF-IDF and FastText in both sentiment analysis and stance detection. However, the overall performance of these models is suboptimal. Random Forest with word2vec achieves the highest macro F1 score of 41.84% in sentiment analysis, while XGBoost with word2vec achieves the highest macro F1 score of 49.34% in stance detection. Despite these results, the scores are still far below 100%, and some models do not surpass the ground truth, indicating significant room for improvement.

RQ2 *What is the impact of Easy Data Augmentation (EDA) on the performance of sentiment analysis and stance detection models?*

While Easy Data Augmentation (EDA) can improve model performance in certain scenarios, it can even reduce model performance. Additionally, it introduces variability and instability across folds in nested cross-validation, leading to inconsistent benefits across different model-embedding combinations.

RQ3 *Do the top-performing models with EDA demonstrate better generalization ability compared to those without EDA?*

Top-performing models with EDA do not generally demonstrate better generalization ability compared to those without EDA. In fact, models without EDA often exhibit smaller performance drops from training to testing or even improved performance on the test set, indicating better generalization ability.

RQ4 *Does the sentiment analysis model with the best generalization ability perform similarly across support, against, and neither stances?*

The sentiment analysis model with the best generalization ability exhibits varied performance across different stances. It shows a bias against the "neither" stance group, while performing similarly on the "support" and "against" stances, highlighting the inherent complexity in the data.

RQ5 *Does the stance detection model with the best generalization ability perform similarly across positive, negative, and neither sentiments?*

The stance detection model with the best generalization ability has a bias against positive sentiment, while it performs similarly in negative and neither sentiments.

RQ6 *Will adding stance labels as additional features in sentiment analysis and adding sentiment labels as additional features in stance detection improve model performance?*

Adding stance labels as additional features in sentiment analysis and adding sentiment labels as additional features in stance detection generally improves model performance. This is evident from the increased average F1 scores and reduced standard deviation. However, the improvements are not statistically significant.

Main RQ *To what extent can machine learning predict the sentiment and stance on PVV-related Reddit comments in the 2023 Dutch general election?*

The overall performance of machine learning on these two tasks was found to be suboptimal. The best sentiment analysis model in test set achieved a macro F1 score of 46.48%, while the best stance detection model in test set achieved a macro F1 score of 46.49%, both significantly lower than the desired 100%. The application of the data augmentation technique, EDA, yielded inconsistent results; some instances demonstrated improvements, whereas others showed declines, with any observed enhancements being statistically insignificant. Furthermore, the models exhibited bias. Even with the inclusion of additional features beyond the text, the improvement in model performance was not significant.

REFERENCES

Akar, Ö., & Güngör, O. (2012). Classification of multispectral images using random forest algorithm. *Journal of Geodesy and Geoinformation*, *1*(2), 105–112.

Al-Rfou, R., Perozzi, B., & Skiena, S. (2013). Polyglot: Distributed word representations for multilingual nlp. *arXiv preprint arXiv:1307.1662*.

Alturayeif, N., Luqman, H., & Ahmed, M. (2023). A systematic review of machine learning techniques for stance detection and its applications. *Neural Computing and Applications*, *35*(7), 5113–5144.

Artetxe, M., Labaka, G., & Agirre, E. (2018). Unsupervised statistical machine translation. *arXiv preprint arXiv:1809.01272*.

Badri, N., Kboubi, F., & Chaibi, A. H. (2022). Combining fasttext and glove word embedding for offensive and hate speech text detection. *Procedia Computer Science*, *207*, 769–778.

Belanger, E., & Aarts, K. (2006). Explaining the rise of the lpf: Issues, discontent, and the 2002 dutch election. *Acta politica*, *41*, 4–20.

Bengio, Y., Ducharme, R., & Vincent, P. (2000). A neural probabilistic language model. *Advances in neural information processing systems*, *13*.

Bestvater, S. E., & Monroe, B. L. (2023). Sentiment is not stance: Target-aware opinion classification for political text analysis. *Political Analysis*, *31*(2), 235–256.

Biau, G., & Scornet, E. (2016). A random forest guided tour. *Test*, *25*, 197–227.

Bishop, C. M., & Nasrabadi, N. M. (2006). *Pattern recognition and machine learning* (Vol. 4). Springer.

Bojanowski, P., Grave, E., Joulin, A., & Mikolov, T. (2017). Enriching word vectors with subword information. *Transactions of the association for computational linguistics*, *5*, 135–146.

Buytaert, M. (2018). *A machine learning approach to sentiment analysis and stance detection for political tweets* [Doctoral dissertation, PhD thesis, Ghent University].

Cambria, E., Das, D., Bandyopadhyay, S., & Feraco, A. (2017). Affective computing and sentiment analysis. *A practical guide to sentiment analysis*, 1–10.

Chaudhry, H. N., Javed, Y., Kulsoom, F., Mehmood, Z., Khan, Z. I., Shoaib, U., & Janjua, S. H. (2021). Sentiment analysis of before and after elections: Twitter data of us election 2020. *Electronics*, *10*(17), 2082.

Chen, J., Tam, D., Raffel, C., Bansal, M., & Yang, D. (2023). An empirical survey of data augmentation for limited data learning in nlp.

Transactions of the Association for Computational Linguistics, *11*, 191–211.

Chen, T., & Guestrin, C. (2016). Xgboost: A scalable tree boosting system. *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*, 785–794.

Chersoni, E., Santus, E., Huang, C.-R., Lenci, A., et al. (2021). Decoding word embeddings with brain-based semantic features. *Computational Linguistics*, *47*(3), 663–698.

da Costa, L. S., Oliveira, I. L., & Fileto, R. (2023). Text classification using embeddings: A survey. *Knowledge and Information Systems*, *65*(7), 2761–2803.

Dang, N. C., Moreno-García, M. N., & De la Prieta, F. (2020). Sentiment analysis based on deep learning: A comparative study. *Electronics*, *9*(3), 483.

Delobelle, P., Winters, T., & Berendt, B. (2020). Robbert: A dutch roberta-based language model. *arXiv preprint arXiv:2001.06286*.

Elghazaly, T., Mahmoud, A., & Hefny, H. A. (2016). Political sentiment analysis using twitter data. *Proceedings of the International Conference on Internet of things and Cloud Computing*, 1–5.

Emanuele, V., De Sio, L., & Van Ditmars, M. (2018). Towards the next dutch general election: Issues at stake, support and priority. *The year of challengers*, 21–26.

Feng, S., Zhou, H., & Dong, H. (2019). Using deep neural network with small dataset to predict material defects. *Materials & Design*, *162*, 300–310.

Gong, X., Ying, W., Zhong, S., & Gong, S. (2022). Text sentiment analysis based on transformer and augmentation. *Frontiers in Psychology*, *13*, 906061.

Grave, E., Bojanowski, P., Gupta, P., Joulin, A., & Mikolov, T. (2018). Learning word vectors for 157 languages. *arXiv preprint arXiv:1802.06893*.

Hastie, T., Tibshirani, R., Friedman, J. H., & Friedman, J. H. (2009). *The elements of statistical learning: Data mining, inference, and prediction* (Vol. 2). Springer.

Hidayatullah, A. F., Cahyaningtyas, S., & Hakim, A. M. (2021). Sentiment analysis on twitter using neural network: Indonesian presidential election 2019 dataset. *IOP conference series: materials science and engineering*, *1077*(1), 012001.

Iyyer, M., Manjunatha, V., Boyd-Graber, J., & Daumé III, H. (2015). Deep unordered composition rivals syntactic methods for text classification. *Proceedings of the 53rd annual meeting of the association for computational linguistics and the 7th international joint conference on natural language processing (volume 1: Long papers)*, 1681–1691.

Khosla, C., & Saini, B. S. (2020). Enhancing performance of deep learning models with different data augmentation techniques: A survey. *2020 International Conference on Intelligent Engineering and Management (ICIEM)*, 79–85.

Kokab, S. T., Asghar, S., & Naz, S. (2022). Transformer-based deep learning models for the sentiment analysis of social media data. *Array*, *14*, 100157.

Korbee, L. (2021). Predicting the 2021 dutch general election outcome through tweet volume and sentiment. *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, *3143*, 3152.

Küçük, D., & Can, F. (2020). Stance detection: A survey. *ACM Computing Surveys (CSUR)*, *53*(1), 1–37.

Küçük, D., & Arıcı, N. (2024). Stance and sentiment analysis of health-related tweets with data augmentation.

Kumar, V., Choudhary, A., & Cho, E. (2020). Data augmentation using pre-trained transformer models. *arXiv preprint arXiv:2003.02245*.

Loria, S., et al. (2018). Textblob documentation. *Release 0.15*, *2*(8), 269.

Melton, C. A., Olusanya, O. A., Ammar, N., & Shaban-Nejad, A. (2021). Public sentiment analysis and topic modeling regarding covid-19 vaccines on the reddit social media platform: A call to action for strengthening vaccine confidence. *Journal of Infection and Public Health*, *14*(10), 1505–1512.

Miao, Z., Li, Y., Wang, X., & Tan, W.-C. (2020). Snippext: Semi-supervised opinion mining with augmented data. *Proceedings of The Web Conference 2020*, 617–628.

Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*.

Mohammad, S., Kiritchenko, S., Sobhani, P., Zhu, X., & Cherry, C. (2016). Semeval-2016 task 6: Detecting stance in tweets. *Proceedings of the 10th international workshop on semantic evaluation (SemEval-2016)*, 31–41.

Mohammad, S. M., Sobhani, P., & Kiritchenko, S. (2017). Stance and sentiment in tweets. *ACM Transactions on Internet Technology (TOIT)*, *17*(3), 1–23.

Mojumder, P., Hasan, M., Hossain, M. F., & Hasan, K. A. (2020). A study of fasttext word embedding effects in document classification in bangla language. *Cyber Security and Computer Science: Second EAI International Conference, ICONCS 2020, Dhaka, Bangladesh, February 15-16, 2020, Proceedings 2*, 441–453.

Otjes, S. (2022). From eurorealism to europhilia? the 2021 dutch elections and the new approach of the netherlands in eu politics. *JCMS: Journal of Common Market Studies*, *60*, 60–75.

Postma, M., Van Miltenburg, E., Segers, R., Schoen, A., & Vossen, P. (2016). Open dutch wordnet. *Proceedings of the 8th Global WordNet Conference (GWC)*, 302–310.

Pradana, R. C., Joddy, S., & Girsang, A. S. (2023). Easy data augmentation for handling imbalanced data in fake news detection. *2023 International Conference on Technology, Engineering, and Computing Applications (ICTECA)*, 1–5.

Prakoso, D. W., Abdi, A., & Amrit, C. (2021). Short text similarity measurement methods: A review. *Soft Computing*, *25*, 4699–4723.

Ramos, J., et al. (2003). Using tf-idf to determine word relevance in document queries. *Proceedings of the first instructional conference on machine learning*, *242*(1), 29–48.

Rodrik, D. (2021). Why does globalization fuel populism? economics, culture, and the rise of right-wing populism. *Annual Review of Economics*, *13*, 133–170.

Rustam, F., Khalid, M., Aslam, W., Rupapara, V., Mehmood, A., & Choi, G. S. (2021). A performance comparison of supervised machine learning models for covid-19 tweets sentiment analysis. *Plos one*, *16*(2), e0245909.

Schmidt, T., Fehle, J., Weissenbacher, M., Richter, J., Gottschalk, P., & Wolff, C. (2022). Sentiment analysis on twitter for the major german parties during the 2021 german federal election. *Proceedings of the 18th Conference on Natural Language Processing (KONVENS 2022)*, 74–87.

Soliman, A., Hafer, J., & Lemmerich, F. (2019). A characterization of political communities on reddit. *Proceedings of the 30th ACM conference on hypertext and Social Media*, 259–263.

Thelwall, M., Buckley, K., Paltoglou, G., Cai, D., & Kappas, A. (2010). Sentiment strength detection in short informal text. *Journal of the American society for information science and technology*, *61*(12), 2544–2558.

Van Heerden, S., & Creusen, B. (2014). Responding to the populist radical right: The dutch case. *European Populism and Winning the Immigration Debate. Brussels: European Liberal Forum & Fores*, 179–210.

Van Holsteyn, J. J. (2018). The dutch parliamentary elections of march 2017. *West European Politics*, *41*(6), 1364–1377.

Vliegenthart, R. (2012). The professionalization of political communication? a longitudinal analysis of dutch election campaign posters. *American Behavioral Scientist*, *56*(2), 135–150.

Wang, C., Nulty, P., & Lillis, D. (2020). A comparative study on word embeddings in deep learning for text classification. *Proceedings of the 4th International Conference on Natural Language Processing and Information Retrieval*, 37–46.

Wei, J., & Zou, K. (2019). Eda: Easy data augmentation techniques for boosting performance on text classification tasks. *arXiv preprint arXiv:1901.11196*.

Wei, W., Zhang, X., Liu, X., Chen, W., & Wang, T. (2016). Pkudblab at semeval-2016 task 6: A specific convolutional neural network system for effective stance detection. *Proceedings of the 10th international workshop on semantic evaluation (SemEval-2016)*, 384–388.

Wirawan, A., Cahyono, H. D., et al. (2023). Easy data augmentation in sentiment analysis of cyberbullying. *2023 6th International Conference on Information and Communications Technology (ICOIACT)*, 443–447.

Yue, T., & Jing, M. (2022). Sentiment analysis based on bert and transformer. *AI and Analytics for Public Health: Proceedings of the 2020 INFORMS International Conference on Service Science*, 317–328.

Zarrella, G., & Marsh, A. (2016). Mitre at semeval-2016 task 6: Transfer learning for stance detection. *arXiv preprint arXiv:1606.03784*.

Zhang, P., Jia, Y., & Shang, Y. (2022). Research and application of xgboost in imbalanced data. *International Journal of Distributed Sensor Networks*, *18*(6), 15501329221106935.

Zhang, X., Zhao, J., & LeCun, Y. (2015). Character-level convolutional networks for text classification. *Advances in neural information processing systems*, *28*.

Zotova, E., Agerri, R., Nunez, M., & Rigau, G. (2020). Multilingual stance detection: The catalonia independence corpus. *arXiv preprint arXiv:2004.00050*.

APPENDIX A: LABELING GUIDELINES FOR SENTIMENT AND STANCE IN PVV-RELATED REDDIT COMMENTS

PROJECT OVERVIEW:

Social media platforms provide huge amounts of unstructured data for different activities to gain insights and get valuable information. This project utilizes data from Reddit to explore the public's sentiment and stance towards the Dutch Party for Freedom (PVV) of the 2023 Dutch election when they won the election and sparked fierce discussion on the Internet. PVV belongs to populist radical right parties and its founder Geert Wilders is controversial for his fierce criticism of Islam.

Sentiment analysis is one of the major text classification tasks in Natural Language Processing. In our task, we define sentiment analysis as: for a piece of text, whether the text producer expresses positive sentiment, negative sentiment, or whether neither sentiment can be inferred regardless of the target.

However, sentiment does not necessarily equate to stance. For stance detection, there are multiple definitions. In our task, we define stance detection as: for an input in the form of a piece of text and a target pair, whether the author is in favour of the given target, against the given target, or whether neither inference is likely.

Consider the target–reddit pair example:

Target: PVV

Reddit: ik heb wel op de PVV gestemd en ik zal je kort vertellen waarom...Er kwamen steeds meer (Noord)Afrikaanse inwoners bij in de wijk, het was zichtbaar dat deze mensen geen baan hadden want ze hingen rond op straat en maakten er een grote teringbende van. Als ik vroeg of ze hun zooi alsjeblieft willen opruimen (inclusief crack pipen, half opgerooke jointies en zelfs nu en dan naalden) was de wereld te klein en kreeg je gelijk ruzie en werden er allerlei verwensingen naar je hoofd geslingerd. (Vaak ook in het Engels aangezien deze mensen de Nederlandse taal niet machtig zijn). Verder werd er steeds meer gesloopt in de wik, de kinderfietsen werden it de achtertuin gestolen en er werd ook steeds vaker it winkels gestolen. Voordat deze mensen in de wijk terecht kwamen hadden we die problemen ook niet, je kon bi wijze van spreken letterlijk je voordeur open laten staan.

We can deduce from the reddit that the speaker directly indicated supporting the target but expresses a negative sentiment by stating lots of frustrating truth happened.

Notice: Stance detection is related to sentiment analysis, but the two have significant differences. In sentiment analysis, systems determine whether a piece of text is positive, negative, or neutral, it may express opinions or sentiment about some other entity. However, in stance detection, systems are to determine the author's favorability towards a given target. In our task, the given target is 'PVV'.

Dataset:

You are provided with two data sets – training set and test set, 410 comments in total for each. The datasets both contain four columns – content, sentiment, stance, and date. We already use keywords select reddits that are related to our topic. The language is Dutch. You are expected to give each row of content two labels - one for sentiment, one for stance.

TASKS There are two tasks:

- **Task A (sentiment labelling):** You will choose one label from "Positive", "Negative", "Neither" for each comment.
- **Task B (stance labelling):** You are expected to choose one label from "Support", "Against", "Neither" for each comment.

Label Principles for Task A:

- **Positive:** We can infer from the text that the author expresses a slight/moderate/strong positive sentiment overall. This could include expressions of joy, approval, support, or optimism regarding different entities.
- **Negative:** We can infer from the text that the author expresses a slight/moderate/strong negative sentiment overall. This includes expressions of anger, disapproval, opposition, or pessimism regarding different entities.
- **Neither:** none of the above.

  If you encounter spam text or only contain website links, just leave the cell blank.

Label Principles for Task B:

- **Target: PVV**
- **FAVOR:** We can infer from the comment that the author supports the target (e.g., directly, or indirectly by supporting someone/something, by opposing or criticizing someone/something opposed to the target, or by echoing the stance of somebody else).
- **AGAINST:** We can infer from the tweet that the tweeter is against the target (e.g., directly, or indirectly by opposing or criticizing someone/something, by supporting someone/something opposed to the target, or by echoing the stance of somebody else).

- **Neither:** none of the above.

  If you encounter spam text or only contain website links, just leave the cell blank.

APPENDIX B

Table 16: Optimal Hyperparameter Sets for Sentiment Analysis Models

EDA	Model Combination	Best Hyperparameter Set
With	LR + TF-IDF	solver: saga, max_iter: 40000, C: 545.5595
	LR + word2vec	solver: lbfgs, max_iter: 40000, C: 206.9138
	LR + FastText	solver: lbfgs, max_iter: 20000, C: 10000.0
	RF + TF-IDF	n_estimators: 48, min_samples_split: 7, min_samples_leaf: 1, max_features: sqrt, max_depth: 39, criterion: entropy, bootstrap: False
	RF + word2vec	n_estimators: 56, min_samples_split: 6, min_samples_leaf: 2, max_features: log2, max_depth: 28, criterion: entropy, bootstrap: False
	RF + FastText	n_estimators: 46, min_samples_split: 8, min_samples_leaf: 1, max_features: sqrt, max_depth: 28, criterion: gini, bootstrap: False
	XGBoost + TF-IDF	subsample: 0.6, max_depth: 10, learning_rate: 0.3, lambda: 0.9, colsample_bytree: 0.8, alpha: 0.2
	XGBoost + word2vec	subsample: 0.5, max_depth: 6, learning_rate: 0.3, lambda: 0.7, colsample_bytree: 0.5, alpha: 0.1
	XGBoost + FastText	subsample: 0.8, max_depth: 9, learning_rate: 0.16, lambda: 0.3, colsample_bytree: 0.7, alpha: 0.3
Without	LR + TF-IDF	solver: lbfgs, max_iter: 20000, C: 0.0006952
	LR + word2vec	solver: saga, max_iter: 30000, C: 29.7635
	LR + FastText	solver: lbfgs, max_iter: 40000, C: 206.9138
	RF + TF-IDF	n_estimators: 92, min_samples_split: 9, min_samples_leaf: 2, max_features: sqrt, max_depth: 14, criterion: gini, bootstrap: False
	RF + word2vec	n_estimators: 83, min_samples_split: 7, min_samples_leaf: 10, max_features: sqrt, max_depth: 37, criterion: gini, bootstrap: False
	RF + FastText	n_estimators: 16, min_samples_split: 2, min_samples_leaf: 8, max_features: sqrt, max_depth: 46, criterion: gini, bootstrap: False
	XGBoost + TF-IDF	subsample: 1.0, max_depth: 8, learning_rate: 0.15, lambda: 0.0, colsample_bytree: 0.5, alpha: 0.7
	XGBoost + word2vec	subsample: 0.6, max_depth: 10, learning_rate: 0.25, lambda: 0.3, colsample_bytree: 0.8, alpha: 0.7
	XGBoost + FastText	subsample: 0.5, max_depth: 5, learning_rate: 0.24, lambda: 0.1, colsample_bytree: 1.0, alpha: 0.2

Table 17: Optimal Hyperparameter Sets for Stance Analysis Models

EDA	Model Combination	Best Hyperparameter Set
With	LR + TF-IDF	solver: saga, max_iter: 40000, C: 1438.44988828766
	LR + word2vec	solver: newton-cg, max_iter: 20000, C: 78.47599703514607
	LR + FastText	solver: lbfgs, max_iter: 20000, C: 10000.0
	RF + TF-IDF	n_estimators: 77, min_samples_split: 7, min_samples_leaf: 1, max_features: sqrt, max_depth: 41, criterion: gini, bootstrap: False
	RF + word2vec	n_estimators: 77, min_samples_split: 7, min_samples_leaf: 1, max_features: sqrt, max_depth: 41, criterion: gini, bootstrap: False
	RF + FastText	n_estimators: 56, min_samples_split: 6, min_samples_leaf: 2, max_features: log2, max_depth: 28, criterion: entropy, bootstrap: False
	XGBoost + TF-IDF	subsample: 0.6, max_depth: 9, learning_rate: 0.19, lambda: 0.1, colsample_bytree: 0.6, alpha: 0.2
	XGBoost + word2vec	subsample: 0.6, max_depth: 8, learning_rate: 0.25, lambda: 0.4, colsample_bytree: 0.8, alpha: 0.1
	XGBoost + FastText	subsample: 0.5, max_depth: 10, learning_rate: 0.23, lambda: 0.5, colsample_bytree: 1.0, alpha: 0.0
Without	LR + TF-IDF	solver: newton-cg, max_iter: 40000, C: 0.23357214690901212
	LR + word2vec	solver: saga, max_iter: 40000, C: 29.763514416313132
	LR + FastText	solver: lbfgs, max_iter: 40000, C: 206.913808111479
	RF + TF-IDF	n_estimators: 73, min_samples_split: 9, min_samples_leaf: 7, max_features: sqrt, max_depth: 14, criterion: entropy, bootstrap: True
	RF + word2vec	n_estimators: 48, min_samples_split: 4, min_samples_leaf: 7, max_features: log2, max_depth: 32, criterion: entropy, bootstrap: True
	RF + FastText	n_estimators: 46, min_samples_split: 9, min_samples_leaf: 2, max_features: log2, max_depth: 4, criterion: entropy, bootstrap: True
	XGBoost + TF-IDF	subsample: 0.6, max_depth: 8, learning_rate: 0.25, lambda: 1.0, colsample_bytree: 0.6, alpha: 0.8
	XGBoost + word2vec	subsample: 0.6, max_depth: 5, learning_rate: 0.13, lambda: 0.1, colsample_bytree: 0.8, alpha: 0.2
	XGBoost + FastText	subsample: 0.5, max_depth: 3, learning_rate: 0.26, lambda: 0.9, colsample_bytree: 0.8, alpha: 0.9

APPENDIX C

Table 18: Ranking of All Models in Sentiment Analysis Based on Average Macro F1 Score

Type	Model	Embedding	Avg. F1 Macro	Std.Dev.	EDA
Sentiment	XGBoost	word2vec	**43.23%**	7.85%	with
	RF	word2vec	41.84%	6.32%	without
	XGBoost	TF-IDF	41.25%	**1.06%**	with
	LR	word2vec	40.51%	3.10%	without
	LR	word2vec	40.24%	5.74%	with
	XGBoost	FastText	39.80%	3.15%	with
	XGBoost	FastText	39.63%	4.44%	without
	LR	FastText	39.61%	5.03%	with
	LR	TF-IDF	39.27%	5.78%	without
	RF	word2vec	38.37%	5.06%	with
	LR	TF-IDF	37.63%	3.06%	with
	XGBoost	word2vec	37.34%	6.51%	without
	RF	TF-IDF	37.30%	2.36%	with
	LR	FastText	37.29%	6.75%	without
	XGBoost	TF-IDF	36.57%	2.55%	without
	RF	FastText	35.52%	3.58%	with
	RF	FastText	35.31%	4.13%	without
	RF	TF-IDF	34.74%	4.28%	without

Table 19: Ranking of All Models in Stance Detection Based on Average Macro F1 Score

Type	Model	Embedding	Avg. F1 Macro	Std.Dev.	EDA
Stance	XGBoost	word2vec	**49.34%**	5.01%	without
	LR	word2vec	49.26%	5.14%	without
	XGBoost	word2vec	48.48%	2.95%	with
	RF	word2vec	48.29%	3.39%	without
	LR	TF-IDF	47.48%	6.18%	with
	RF	word2vec	45.90%	4.32%	with
	LR	TF-IDF	45.60%	5.50%	without
	LR	word2vec	44.32%	8.06%	with
	LR	FastText	44.23%	6.94%	without
	RF	FastText	43.50%	5.47%	with
	XGBoost	FastText	43.22%	3.73%	with
	XGBoost	TF-IDF	43.07%	5.34%	with
	RF	FastText	42.93%	**2.13%**	without
	XGBoost	TF-IDF	41.89%	4.57%	without
	RF	TF-IDF	41.46%	3.57%	with
	LR	FastText	40.98%	3.80%	with
	RF	TF-IDF	39.99%	3.90%	without
	XGBoost	FastText	39.14%	2.38%	without

APPENDIX D

Table 20: Sample Train Data before Data Preprocessing

Content	Sentiment	Stance
Om het nog bondiger te zeggen: ik stemde ooit links en toen ging ik in een van die wijken wonen waar Geert Wilders ook ooit heeft gewoond en nu stem ik PVV. Daar waren jaren aan haat voor nodig, haat die ik ontving maar niet uitzond. Alle pogingen tot vriendelijkheid, tot verbinden, werden beloond met haat. Ik gaf het op. Er is iets grondig mis met de Islamitische gemeenschap en acceptatie werd niet als 1 richting verkeer.	Negative	Support
Volgens de peilingen stond PVV 2 weken geleden nog op 18 zetels. Dat ze nu opeens op 35 staan is inderdaad wel een beetje een schok.	Negative	Neither
PVV stemmers mogen dan het geheugen hebben van een goudvis, ik heb dat niet. Daarnaast komt de leus uit 2014 en is hij er voor veroordeeld in 2020 waarna hij er geen afstand van nam.	Negative	Against
PVV wil geen vluchtelingen, Omtzigt wil geen expats en de VVD wil eigenlijk alles zo houden zoals het is, maar dan wel met harde taal.	Neither	Neither
Dat weten we niet. PVV heeft veel strategische stemmen gekregen. Die kunnen zo weer naar iemand anders.	Neither	Against
Het klopt dat PVV moet gaan samenwerken met andere partijen maar dat betekent niet dat dit land niet gaat veranderen. Klimaatdoelen gaan het raam uit, we slaan internationaal en flater en de intolerantie voor mensen die anders zijn dan Henk en Ingrid zal groeien.	Negative	Against
Fvd is nu toch wel een wappie partij dus gaat iedereen weer terug naar de OG Geert?	Negative	Against